

核心解释（小程序端生效版 `:deep()` 写法）

你这个能生效的写法是 **SCSS 嵌套格式的深度选择器**，结合 `!important` 强制提升样式优先级，专门适配小程序的样式解析规则，拆解如下：

1. 逐部分含义

代码片段	作用
<code>:deep()</code>	深度选择器（Vue 专属），突破 <code><style scoped></code> 的样式隔离，能修改子组件（如 <code>uni-icons</code> ）的样式； ⚠ 小程序端对 <code>:deep()</code> 空括号+内部嵌套的写法解析更友好，比 <code>:deep(.uni-icons)</code> 兼容性更高。
<code>.uni-icons</code>	选中 <code>uni-icons</code> 组件的根元素（小程序内置图标组件的核心类名）。
<code>!important</code>	强制提升样式优先级，覆盖 <code>uni-icons</code> 组件内部自带的 <code>color</code> 样式（组件源码大概率给 <code>color</code> 加了高优先级样式）。
<code>\$brand-theme-color</code>	项目预定义的 SCSS 变量（品牌主题色），最终会编译成具体的颜色值（如 <code>#1890ff</code> ）。

2. 为什么这个写法在小程序生效？

- **语法适配：**小程序对 `:deep()` 空括号 + 内部嵌套类名的解析规则更兼容，而 `:deep(.uni-icons)` 这种“括号内直接写类名”的写法，在小程序编译时会被简化，权重不足；
- **优先级足够：**`!important` 是 CSS 最高优先级标记，能直接覆盖 `uni-icons` 组件内部的默认样式（哪怕组件里也用了 `!important`，后加载的样式会生效）；

- **突破样式隔离：** `<style scoped>` 会给当前页面样式加唯一标识，默认改不了子组件样式，`:deep()` 就是用来穿透这个隔离，让样式能作用到 `uni-icons` 组件上。

3. 简化管理

这句话的核心作用：**强制修改小程序里 `uni-icons` 图标组件的颜色为项目主题色，且能百分百生效。**

4. 小提醒

- 这个写法会修改页面中**所有** `uni-icons` 的颜色，如果只想改某个图标，建议给图标加自定义类名：

Code block

```
1  :deep() {  
2    .uni-icons.custom-icon { // 只改带 custom-icon 类的图标  
3      color: $brand-theme-color !important;  
4    }  
5  }
```

- 小程序端修改内置组件样式，`!important` 几乎是必须的，不用纠结，只要缩小作用域（如加自定义类）就不会污染全局。

（注：文档部分内容可能由 AI 生成）